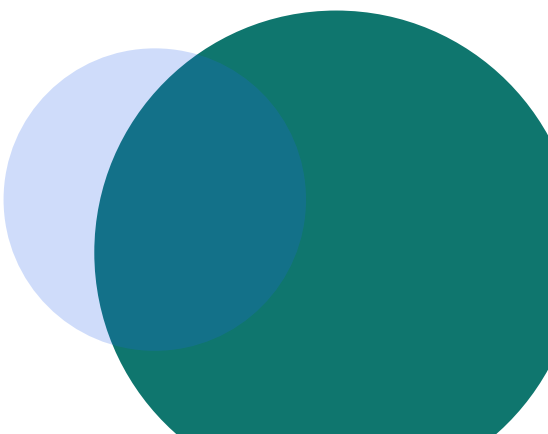
Purse Autonomous Finance

Technical Design Document

Architecture, account model, agent cognition, and resilience

Version	2.4.0
Status	Production
Target environment	AI Studio Edge Runtime and modern web browsers
Repository source	Imported from `dwmk/purse.ai`



Prepared as a structured LaTeX edition of the supplied Markdown TDD.

Contents

1 Executive Technical Summary **1**

2 High-Level Architecture **1**

2.1 Architecture diagram . . . 1

2.2 Client application . . . 2

2.3 Serverless proxy layer . . . 3

2.4 External AI provider APIs . . . 3

3 Account Subsystem Architecture **3**

3.1 Account data model . . . 3

3.2 Account type matrix and defaults . . . 3

3.3 Default cash account philosophy . . . 4

4 LLM Cognitive Engine and Fiona Persona **4**

4.1 Multi-LLM orchestration . . . 4

4.2 Generic query versus transactional logic . . . 4

4.3 Safe execution proposal gate schema . . . 5

4.3.1 Expense transaction example . . . 5

4.3.2 New crypto account example . . . 6

4.3.3 Transfer example . . . 6

5 Security and Edge Resilience **6**

6 Verification and Build Health **7**

1 Executive Technical Summary

Purse is a decentralized, client-first autonomous personal finance copilot. It pairs an intelligent avatar agent, **Fiona**, with an in-browser financial core. Its stated architectural paradigm is **Edge-First Local-First**: financial records and agent memory remain in browser persistence rather than in a remote multi-tenant financial database.

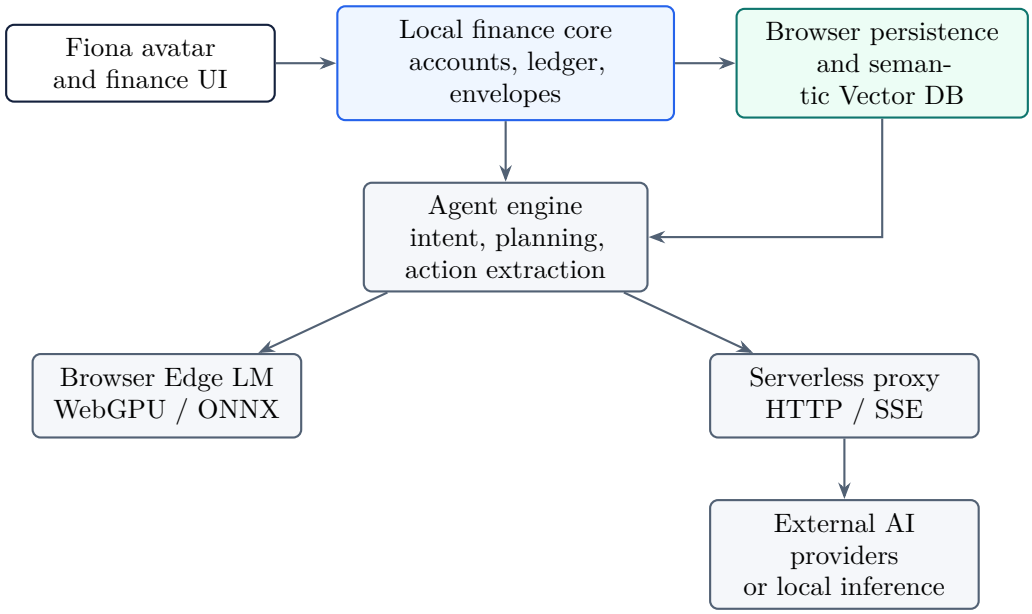


Figure 1: Conceptual view of the client-first finance and cognition architecture.

The system characteristics described in the source are:

- Financial balances, ledger transactions, budget envelopes, and agent memory graphs are held in browser persistence using `localStorage`, `CacheStorage`, and an in-browser semantic vector database.
- High-reasoning tasks use the user's selected LLM: Google Gemini, OpenAI GPT-4o, DeepSeek Chat, Zhipu GLM-4, or an offline browser model using `@huggingface/transformers`, WebGPU, and ONNX.
- The agent distinguishes general conversational questions from operational instructions that may change accounts, cash flow, budgets, bills, or memory rules.
- Account types include cash, crypto wallets, checking, savings, credit cards, and investment/brokerage accounts.

2 High-Level Architecture

2.1 Architecture diagram

Figure 2 expands the application into its client, serverless proxy, and external AI-provider layers. The drawing is a visual reorganization of the architecture in the source document; component names and responsibilities are retained.

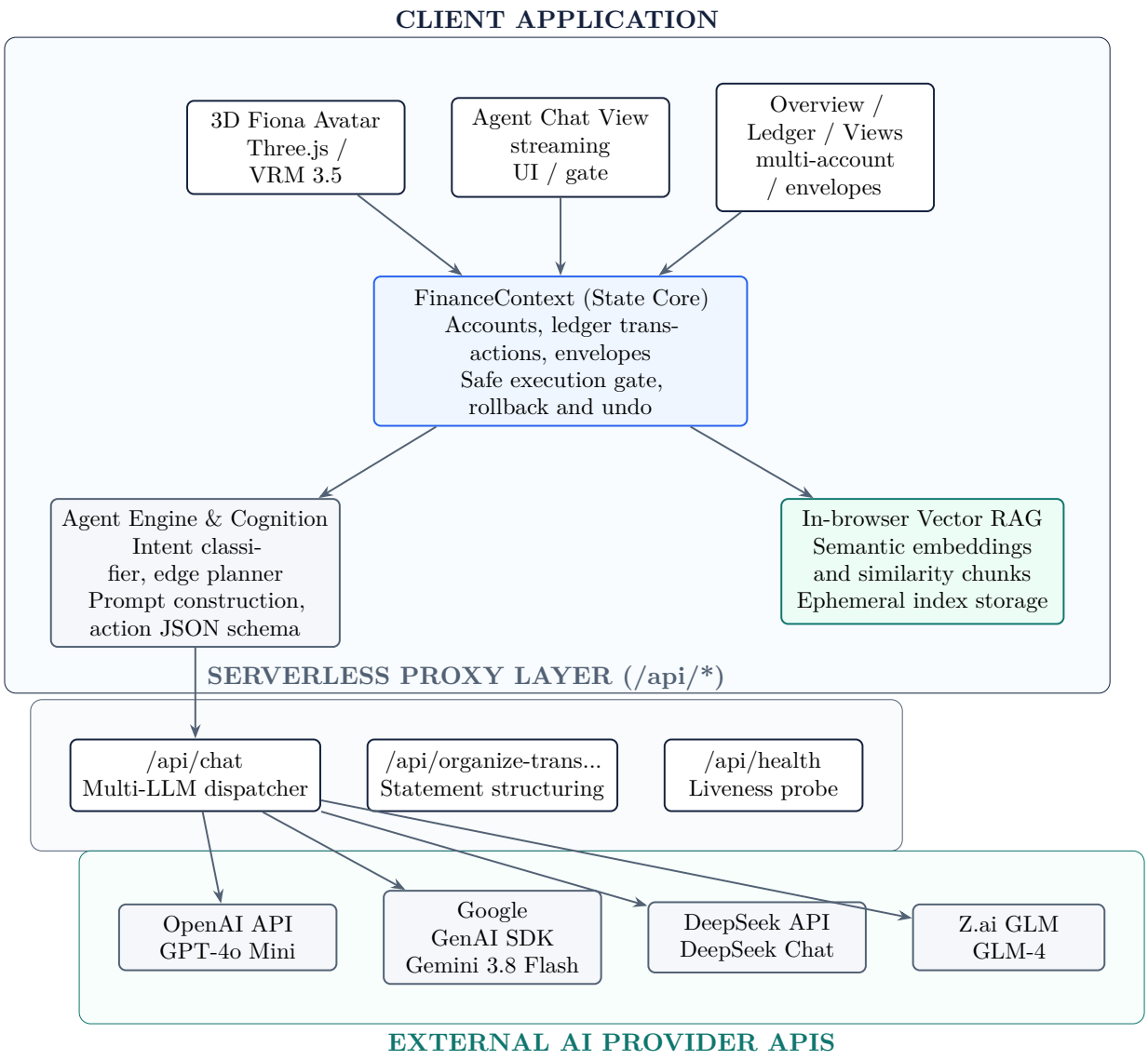


Figure 2: High-level architecture: client application, serverless proxy endpoints, and AI providers.

2.2 Client application

The client includes:

- **3D Fiona Avatar:** Three.js / VRM 3.5.
- **Agent Chat View:** streaming UI and execution gate.
- **Overview, Ledger, and Views:** multi-account and envelope presentation.
- **FinanceContext:** account state, ledger transactions, envelopes, safe execution proposals, rollback stack, and undo engine.
- **Agent Engine and Cognition:** intent classification, edge planning, system-prompt construction, action JSON extraction, and schema handling.
- **In-browser Vector RAG:** semantic embeddings, vector similarity chunks, and ephemeral index storage.

2.3 Serverless proxy layer

The source identifies three endpoints:

- `/api/chat`: multi-LLM dispatcher.
- `/api/organize-trans...`: statement structuring (endpoint name is abbreviated in the source).
- `/api/health`: liveness probe.

The client communicates through an HTTP / Server-Sent Events (SSE) stream.

2.4 External AI provider APIs

The architecture diagram lists Google GenAI SDK (Gemini 3.8 Flash), OpenAI API (GPT-4o Mini), DeepSeek API (DeepSeek-Chat), and Z.ai GLM (GLM-4).

3 Account Subsystem Architecture

3.1 Account data model

The account model is defined in `src/types/finance.ts`.

Listing 1: Account and account-type declarations

```
export type AccountType =
  | 'cash' | 'crypto' | 'checking'
  | 'savings' | 'credit' | 'investment';

export interface Account {
  id: string;
  name: string;
  type: AccountType;
  balance: number;
  currency: string;
  institution?: string;        // Optional; omitted for unbanked cash
  network?: string;            // e.g. Solana, Ethereum, Bitcoin, Polygon
  walletAddress?: string;      // e.g. 0x71C...3F8 or 9WzD...xyz
  accountNumberMask: string; // e.g. Cash, Web3, **** 4412
  lastUpdated: string;
}
```

3.2 Account type matrix and defaults

Table 1: Account types, defaults, identifiers, and use cases.

Type	Default name	Institution requirement	Key identifiers	Typical use case
cash (default)	Physical Cash Wallet	None (`""` or omitted)	`accountNumberMask: "Cash"`	Pocket cash, envelope cash, tips, petty cash

Type	Default name	Institution requirement	Key identifiers	Typical use case
crypto	Web3 Crypto Wallet	Self-Custody or wallet name	`network`, `walletAddress`	Ethereum, Solana, Bitcoin, hardware wallets, DeFi pools
checking	Primary Operating Checking	Bank / credit union	Masked account number (•••• `1234`)	Daily expenses, payroll deposits
savings	Emergency Savings Vault	Bank / financial institution	Masked account number (•••• `5678`)	Liquidity reserve, zero-income runway floor
credit	Rewards Credit Card	Card issuer	Masked card number (•••• `9012`)	Revolving debt and credit liabilities
investment	Stock Brokerage	Brokerage custodian	Masked account number (•••• `3456`)	ETFs, equities, index funds, long-term capital

3.3 Default cash account philosophy

The system defaults to an unbanked **Physical Cash Wallet** to avoid requiring bank names, routing numbers, or third-party OAuth links. Its starting balance defaults to `0.00` or a user-defined amount. Cash is treated as a liquid asset alongside checking accounts when calculating **Safe Daily Spend** and **Zero-Income Runway**.

4 LLM Cognitive Engine and Fiona Persona

4.1 Multi-LLM orchestration

The cognitive core is located in `src/services/agentEngine.ts`. The source describes the following inference options.

Table 2: LLM providers and inference modes described in the source.

Provider / mode	Model	Integration notes
Google Gemini	`gemini-3.8-flash`	Default; server-side proxy with client API-key passthrough or environment fallback; SSE token streaming.
OpenAI	`gpt-4o-mini` / `gpt-4o`	REST streaming and completion proxy.
DeepSeek	`deepseek-chat`	High-efficiency reasoning.
Z.ai	`glm-4-flash`	Multilingual Asian and international reasoning.
Browser Edge LM	`smolLM2-135M-instruct`	In-browser inference using WebGPU / ONNX with no external API calls.

4.2 Generic query versus transactional logic

Fiona is described as a knowledgeable, empathetic, human-like personal secretary. The processing flow is shown in Figure 3.

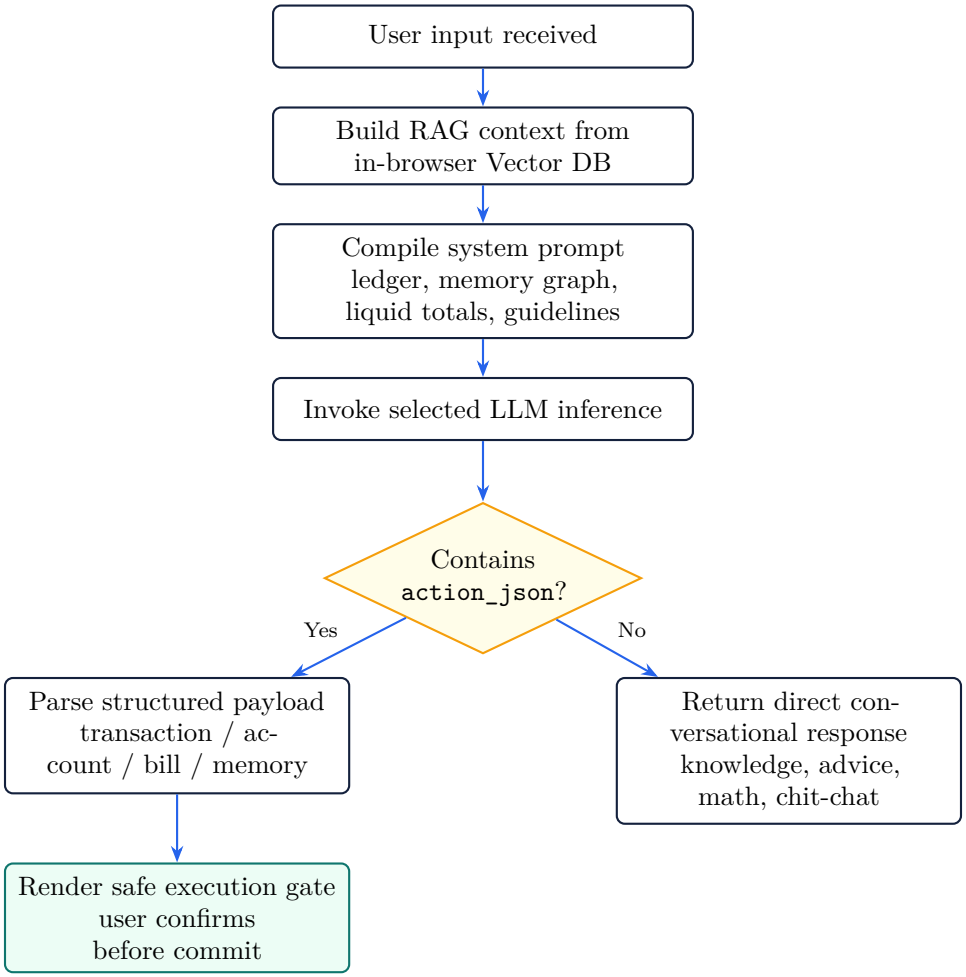


Figure 3: Fiona's conversational and transactional decision flow.

When an action payload is present, the source lists these action families:

- `ADD_TRANSACTION`: income, expense, or transfer.
- `ADD_ACCOUNT`: cash, crypto, or bank account.
- `ADD_BILL`: scheduled payments.
- `UPDATE_MEMORY`: memory-rule invariants.

General questions are answered directly in Fiona's persona, without being routed through a transactional action.

4.3 Safe execution proposal gate schema

When the LLM proposes changes to financial accounts or records, it emits an `action_json` fenced payload. The application renders a safe execution gate, and the user confirms before the change is committed.

4.3.1 Expense transaction example

Listing 2: Example ADD_TRANSACTION payload for an expense

```
{
```

```
  "action": "ADD_TRANSACTION",
  "type": "expense",
  "amount": 25.50,
  "accountId": "acc_cash_01",
  "category": "Dining & Nightlife",
  "merchant": "Blue Bottle Coffee",
  "note": "Afternoon latte"
}
```

4.3.2 New crypto account example

Listing 3: Example ADD_ACCOUNT payload

```
{
  "action": "ADD_ACCOUNT",
  "name": "Phantom Solana Wallet",
  "type": "crypto",
  "balance": 350.00,
  "currency": "USD",
  "institution": "Self-Custody",
  "network": "Solana",
  "walletAddress": "9WzD...xyz"
}
```

4.3.3 Transfer example

Listing 4: Example transfer payload

```
{
  "action": "ADD_TRANSACTION",
  "type": "transfer",
  "amount": 100.00,
  "fromAccountId": "acc_chk_01",
  "toAccountId": "acc_cash_01",
  "merchant": "ATM Cash Withdrawal",
  "category": "Other",
  "note": "Withdrew cash for weekly spending"
}
```

5 Security and Edge Resilience

The source identifies four resilience and privacy mechanisms.

1. **Client-side data isolation.** Ledger entries and credentials reside in the user's browser. Serverless functions proxy AI calls without logging payload data.
2. **Deterministic fallbacks.** If an external API is unavailable or quota-limited, `agentEngine.ts` runs `runEdgeCognition` heuristics to calculate runway, detect subscription leaks, and format responses.
3. **Rollback and invariant stack.** Every applied action enters an undo stack. Transactions can be reverted with one click, restoring balances to their prior states.

4. **Edge vectors.** Vector RAG runs client-side, avoiding external vector-hosting costs.

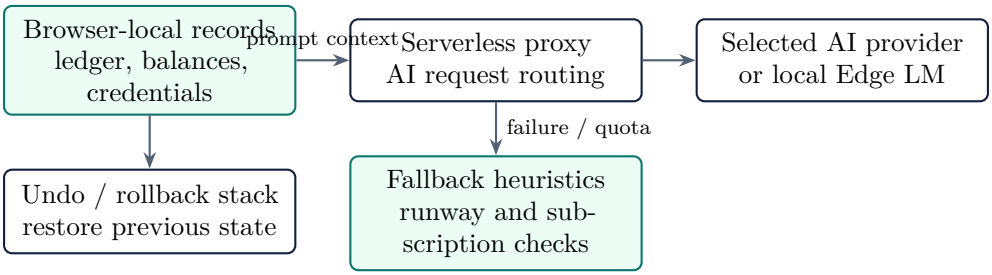


Figure 4: Privacy and resilience concepts described in the source TDD.

6 Verification and Build Health

The supplied TDD reports the following checks as passing:

- Full TypeScript compilation passes with zero warnings.
- Serverless API handlers mount properly in `server.ts` and the Vite development server.
- `npm run build` compiles clean production bundles.

Document Notes

This edition reorganizes the supplied Markdown into a typeset technical document, with tables, code listings, and TikZ diagrams. Technical statements and implementation details are based on the source file; no external verification has been added. The abbreviated endpoint `/api/organize-trans...` is retained as supplied.